

GOLD-001 — Batch 005

Source-Integrity Review & Owner QC Packet

Production RAG v1 · evidence-candidate authoring · prepared 2026-08-23T16:43:37Z · 19 decisions outstanding

AN INTERNAL AUTHORIZING REVIEW IS NOT INDEPENDENT VERIFICATION

The statuses in this document were produced by the authoring model reading its own output against the frozen evidence. That is a self-check. A `READY_FOR_OWNER_REVIEW` label means the author found nothing wrong — which is exactly the claim an independent reviewer exists to test.

Nothing here is verified. All 19 candidates are `candidate_unverified`, all 19 decisions are `null`, and the project's confirmed total is unchanged at **67** holdout-eligible cases from batches 001–004.

19

candidates awaiting an owner decision

7 / 4

repaired / recommended for rejection

19 / 19

precheck holdout-ready before and after review

0

human_verified nothing is gold

1. What the review found

All 19 candidates were `precheck_holdout_ready` before this review and all 19 still are. The review nonetheless recommends **4** for rejection and repaired **7**. That gap is the same point batch 004 made and this batch confirms with a second sample: a structural check verifies hashes, offsets and string containment, and cannot read.

id	provider	reasoning type	internal status	repaired	findings
01	anthropic	ambiguity_disambiguation	REJECT_RECOMMENDED	no	4
02	anthropic	configuration_interaction	READY_FOR_OWNER_REVIEW	no	1
03	anthropic	error_behavior	READY_FOR_OWNER_REVIEW	no	2
04	anthropic	error_behavior	READY_FOR_OWNER_REVIEW	no	2
05	anthropic	exact_lookup	NEEDS_REPAIR	yes	2
06	anthropic	lifecycle_compatibility_migration	REJECT_RECOMMENDED	no	3
07	anthropic	lifecycle_compatibility_migration	READY_FOR_OWNER_REVIEW	no	1
08	anthropic	lifecycle_compatibility_migration	NEEDS_REPAIR	yes	2
09	openai	error_behavior	NEEDS_REPAIR	yes	3
10	openai	configuration_interaction	REJECT_RECOMMENDED	no	3
11	openai	configuration_interaction	NEEDS_REPAIR	yes	3
12	openai	configuration_interaction	READY_FOR_OWNER_REVIEW	no	1
13	openai	configuration_interaction	REJECT_RECOMMENDED	no	3
14	openai	configuration_interaction	READY_FOR_OWNER_REVIEW	no	1
15	openai	configuration_interaction	NEEDS_REPAIR	yes	2

id	provider	reasoning type	internal status	repaired	findings
16	openai	configuration_interaction	NEEDS_REPAIR	yes	2
17	openai	error_behavior	READY_FOR_OWNER_REVIEW	no	1
18	openai	exact_lookup	NEEDS_REPAIR	yes	2
19	openai	lifecycle_compatibility_migration	READY_FOR_OWNER_REVIEW	no	1

Recommended for rejection

candidate	reason
GOLD-B005-01	recommended for rejection: an error-code value documented twice, presented as scoped ambiguity
GOLD-B005-06	recommended for rejection: a schema field description mined as a lifecycle statement
GOLD-B005-10	recommended for rejection: reversed relation, and the subject's identity is outside the anchor
GOLD-B005-13	recommended for rejection: the same relation as B005-12, from the adjacent sentence

These are recommendations, not decisions. Nothing binds the owner, and a rejection recommendation the owner disagrees with is a candidate the owner may approve.

2. Defects in the generator, not just in the candidates

A review that reports only candidate-level findings is a review that was not looking at the thing that produced them. Three defects surfaced here, all recorded rather than patched: the batch-005 generation artifact is not being regenerated, and a fix belongs in batch 006's preregistration where it can be declared before it sees a candidate.

- **the bare-definition-bullet rule only inspects single-span records** — seen in GOLD-B005-01 . The generation self-review drops a candidate whose single span is a bare '- field : description' bullet, because its scope lives in the heading. B005-01 is two such bullets in a multi-span record, and the rule's `len(spans) == 1` guard let it through. Recorded rather than patched: batch 005's generation artifact is not being regenerated, and a fix belongs in batch 006's preregistration.
- **markdown reference links survive into questions built from conditional sentences** — seen in GOLD-B005-15 . The link stripper runs on the composed question, but a reference-style link whose label is itself backticked was not matched. Repaired here by hand; the pattern should be fixed before batch 006.
- **prose mistaken for a section heading by the parser** — seen in GOLD-B005-11 . `section_path` reads 'configured through AWS_REGION, AWS_DEFAULT_REGION, or your AWS profile.' No claim depends on it, and closed batches must not be touched, so this is recorded as a parser observation for a future audit.

3. The nineteen candidates

Each card carries the final question, answer and claims, the exact evidence with its hash, what the review found, and what it changed. This is the material to check.

GOLD-B005-01 · REJECT_RECOMMENDED

anthropic · Web fetch tool · Response › Errors
reasoning type `ambiguity_disambiguation` · shape `multi_document` · precheck holdout-ready True

Q. In Web fetch tool, what does the `invalid_tool_input` field mean, and how does that differ from Tool search tool?

A. In Web fetch tool, `invalid_tool_input` is: Invalid tool input, such as a malformed URL or a non-HTTP(S) scheme. In Tool search tool, `invalid_tool_input` is: the search input was invalid, for example a malformed regex pattern or a pattern over the 200-character limit.

ATOMIC CLAIMS

1. In Web fetch tool, `invalid_tool_input` is: Invalid tool input, such as a malformed URL or a non-HTTP(S) scheme.
2. In Tool search tool, `invalid_tool_input` is: the search input was invalid, for example a malformed regex pattern or a pattern over the 200-character limit.

EXACT EVIDENCE

E1 · VER_901356D3FFCE0F0478BA2D33AEFDF98A 22545–22636 (91 CHARS) · RESPONSE › ERRORS

* ``invalid_tool_input``: Invalid tool input, such as a malformed URL or a non-HTTP(S) scheme

critical strings: `invalid_tool_input`
evidence_hash `ee532724156923f6591fd9e72285493442b2e0e15541ce46c2f879c5ba54458c`

E2 · VER_B7EA8359F97CA269418988F78E80B870 28727–28860 (133 CHARS) · ERROR HANDLING › TOOL RESULT ERRORS (200 STATUS)

* ``invalid_tool_input``: the search input was invalid, for example a malformed regex pattern or a pattern over the 200-character limit

critical strings: `invalid_tool_input`
evidence_hash `caf2d8758e197a6698a3f58e49adee4ccdf7bd10082018815144c119cc72ffb3`

INTERNAL REVIEW FINDINGS

- NOT_A_FIELD: `invalid_tool_input` is a value of the `error_code` field, not a field. The question's noun is wrong before anything else is considered.
- NOT_AMBIGUITY: the two readings are the same concept — 'the input you gave this tool was invalid' — with tool-specific examples (a malformed URL for web fetch, a malformed regex or an over-length pattern for tool search). §12 asks for a term whose semantics differ by scope; an error code whose examples differ by tool is one meaning documented twice.
- SCOPE_IN_HEADING: neither span mentions its tool. E1 reads '* `invalid_tool_input` : Invalid tool input, such as a malformed URL...' and E2 '* `invalid_tool_input` : the search input was invalid...'; 'Web fetch tool' and 'Tool search tool' come from the document titles. This is the bare-definition-bullet defect the generator drops elsewhere — it did not fire here because the rule only inspects single-span records, which is a generator bug recorded below.
- S3 is explicit that ambiguity must not be preserved because the batch has only one such case.

GOLD-B005-02 · READY_FOR_OWNER_REVIEW

anthropic · Admin · Federation Issuers › Create Federation Issuer

reasoning type `configuration_interaction` · shape `single_span` · precheck holdout-ready True

Q. What happens when `jwtks.type` is `discovery` and no `discovery_base` is set?

A. The issuer URL must be publicly reachable over HTTPS so Anthropic can fetch the discovery document; for `explicit_url` and `inline` modes the issuer URL is only matched as the JWT's `iss` claim and is not fetched.

ATOMIC CLAIMS

1. When `jwtks.type` is `discovery` and no `discovery_base` is set, the issuer URL must be publicly reachable over HTTPS so Anthropic can fetch the discovery document; for `explicit_url` and `inline` modes the issuer URL is only matched as the JWT's `iss` claim and is not fetched.

INTERACTION RECORDED

- **A:** `jwtks.type` set to `discovery`
- **B:** `discovery_base` absent
- **relation:** with A selected and B unset, the issuer URL must be publicly reachable over HTTPS so the discovery document can be fetched; under `explicit_url` and `inline` the URL is matched as the `iss` claim and never fetched

EXACT EVIDENCE

E1 · VER_C299B58FE1F5A4D3A081B550334A7DF6 469462-469739 (277 CHARS) · FEDERATION ISSUERS ›
CREATE FEDERATION ISSUER

```
When `jwtks.type` is
`discovery` and no `discovery_base` is set, the issuer URL must be
publicly reachable over HTTPS so Anthropic can fetch the discovery
document; for `explicit_url` and `inline` modes the issuer URL is only
matched as the JWT's `iss` claim and is not fetched.
```

critical strings: `jwtks.type`, `discovery`, `discovery_base`

evidence_hash a2602d47d43f5d8dd96abef379a50a1cfd6144349d27e7082f1ae8a69862d873

INTERNAL REVIEW FINDINGS

- The anchor carries all three parts §5 asks for: `jwtks.type` is `discovery`, no `discovery_base` is set, and the resulting requirement. No boundary repair is needed.

GOLD-B005-03 · READY_FOR_OWNER_REVIEW

anthropic · Beta · Deployment Runs › List Deployment Runs › Query Parameters
reasoning type `error_behavior` · shape `single_span` · precheck holdout-ready True

Q. What happens when filtering by a non-existent `deployment_id`?

A. It returns 200 with empty data.

ATOMIC CLAIMS

1. Filtering by a non-existent `deployment_id` returns 200 with empty data.

EXACT EVIDENCE

E1 · VER_DE7F74230C8F10D30AEA5D037A3BD0A5 2711799–2711869 (70 CHARS) · DEPLOYMENT RUNS › LIST DEPLOYMENT RUNS › QUERY PARAMETERS

Filtering by a non-existent `deployment_id` returns 200 with empty data.

critical strings: `200 with empty data`

evidence_hash 892ee8dfcbdd186e6454dbbed9e5f70ba5c5f353b262c62c1749656a6d93bd63

INTERNAL REVIEW FINDINGS

- Condition and result are both inside the anchor, as §6 requires.
- NONCRITICAL_SCOPE: the span does not name the endpoint whose list behaviour this is. The question is answerable without it, so the anchor stands; an owner who wants the endpoint named should choose NEEDS_EDIT.

GOLD-B005-04 · READY_FOR_OWNER_REVIEW

anthropic · Web search tool · Response › Search results
reasoning type `error_behavior` · shape `single_span` · precheck holdout-ready True

Q. What happens if `encrypted_content` is missing or modified?

A. The request fails with a 400 validation error.

ATOMIC CLAIMS

1. If `encrypted_content` is missing or modified, the request fails with a 400 validation error.

EXACT EVIDENCE

E1 · VER_53DA2F78E855C75EC755089C13D44C28 20614–20707 (93 CHARS) · RESPONSE › SEARCH RESULTS

If ``encrypted_content`` is missing or modified, the request fails with a 400 validation error.

critical strings: `encrypted_content`

evidence_hash bf645608c00bf09471aa9890464ff50f4ed0500135209a9ceab88a6e0b9d29db

INTERNAL REVIEW FINDINGS

- Condition and result are both inside the anchor.
- NONCRITICAL_SCOPE: the span does not say that `encrypted_content` belongs to the web search tool's results. The fact is answerable as asked.

GOLD-B005-05 · NEEDS_REPAIR

anthropic · Server tools · Domain filtering

reasoning type `exact_lookup` · shape `single_span` · precheck holdout-ready True

Q. What constraint does a request-level `allowed_domains` list have to satisfy?

A. Request-level `allowed_domains` must be a subset of the organization-level allowed list; entries outside it cause the API to return a validation error.

ATOMIC CLAIMS

1. Request-level `allowed_domains` must be a subset of the organization-level allowed list; entries outside it cause the API to return a validation error.

EXACT EVIDENCE

E1 · VER_8D2A22E3827C98E0B9D4E1EF411E5353 40135-40286 (151 CHARS) · DOMAIN FILTERING

Request-level `allowed_domains` must be a subset of the organization-level allowed list; entries outside it cause the API to return a validation error.

critical strings: `allowed_domains`

evidence_hash `e46486f298067b5585bf926b4dffbf606adc0fabe989a04f569377183b1da3a5`

INTERNAL REVIEW FINDINGS

- QUESTION_SCOPE: 'What must `allowed_domains` be?' drops the qualifier the evidence itself supplies. The span constrains a `*request-level*` `allowed_domains` list relative to the organization-level list; asked unscoped, the question implies a rule about the parameter in general.
- The scope word is inside the anchor, so this is a question repair and the evidence does not move.

REPAIRS MADE

- **question rewritten**

was: What must `allowed_domains` be?

now: What constraint does a request-level `allowed_domains` list have to satisfy?

GOLD-B005-06 · REJECT_RECOMMENDED

anthropic · Beta · Models › List Models › Returns

reasoning type `lifecycle_compatibility_migration` · shape `single_span` · precheck holdout-ready True

Q. Where is `fallbacks` supported?

A. Model IDs this model accepts as `fallbacks[i].model` on the Messages API. An empty list means the `fallbacks` parameter is not supported for this model as primary.

ATOMIC CLAIMS

1. Model IDs this model accepts as `fallbacks[i].model` on the Messages API. An empty list means the `fallbacks` parameter is not supported for this model as primary.

EXACT EVIDENCE

E1 · VER_DE7F74230C8F10D30AEA5D037A3BD0A5 8864–9027 (163 CHARS) · MODELS › LIST MODELS › RETURNS

```
Model IDs this model accepts as `fallbacks[i].model` on the Messages API. An empty list means the
`fallbacks` parameter is not supported for this model as primary.
```

critical strings: `fallbacks`

evidence_hash 0233ca0ebe16c41bd827639d7138b79a98ca3cc5b357e7d6d75bd908bbe774ca

INTERNAL REVIEW FINDINGS

- **NOT_A_LIFECYCLE_STATEMENT:** the span is a schema field description from a List Models response — 'Model IDs this model accepts as `fallbacks[i].model` on the Messages API' — not a statement about support status. It was mined as lifecycle because its second sentence contains 'not supported'.
- **UNANSWERABLE_AS_ASKED:** 'Where is `fallbacks` supported?' asks for a set of models. The span says only that an empty list means the parameter is unsupported for that model as primary, which does not answer it.
- A repair would have to rewrite the question into a different fact; the honest outcome is rejection.

GOLD-B005-07 · READY_FOR_OWNER_REVIEW

anthropic · Context editing · Client-side compaction (SDK)

reasoning type `lifecycle_compatibility_migration` · shape `single_span` · precheck holdout-ready True

Q. Is `compaction_control` still supported?

A. The `compaction_control` parameter is deprecated in the Python, TypeScript, and Ruby SDKs and will be removed in a future version.

ATOMIC CLAIMS

1. The `compaction_control` parameter is deprecated in the Python, TypeScript, and Ruby SDKs and will be removed in a future version.

EXACT EVIDENCE

E1 · VER_1C53B961E1F5DA8124A1E7E8EB92C941 75250–75380 (130 CHARS) · CLIENT-SIDE COMPACTION (SDK)

The ``compaction_control`` parameter is deprecated in the Python, TypeScript, and Ruby SDKs and will be removed in a future version.

critical strings: `compaction_control`

evidence_hash `b2f29d24dba4045e411f87f438b77bc9271fe24588f500c999d148195d44736a`

INTERNAL REVIEW FINDINGS

- Self-contained: names the parameter, the three SDKs, the deprecation, and the future removal. Narrowest accurate type is deprecation, which is what the secondary category records.

GOLD-B005-08 · NEEDS_REPAIR

anthropic · Prompting Claude Sonnet 5 · Calibrating effort and thinking depth

reasoning type lifecycle_compatibility_migration · shape single_span · precheck holdout-ready True

Q. Is manual extended thinking with `budget_tokens` supported on Claude Sonnet 5?

A. Manual extended thinking (`thinking: {type: "enabled", budget_tokens: N}`) is not supported on Claude Sonnet 5 and returns a 400 error.

ATOMIC CLAIMS

1. Manual extended thinking (`thinking: {type: "enabled", budget_tokens: N}`) is not supported on Claude Sonnet 5 and returns a 400 error.

EXACT EVIDENCE

E1 · VER_9C5166B670BF43589EE63D0DBE8B93D2 5356–5491 (135 CHARS) · CALIBRATING EFFORT AND THINKING DEPTH

```
Manual extended thinking (`thinking: {type: "enabled", budget_tokens: N}`) is not supported on Claude Sonnet 5 and returns a 400 error.
```

critical strings: `budget_tokens`

evidence_hash 58fbbedd718a3c5c6487312de617ed46995249883d398207176d61663dda9dbbe

INTERNAL REVIEW FINDINGS

- **QUESTION_FORM:** 'Where is `budget_tokens` supported?' asks for the set of places it works. The evidence states one place it does not — manual extended thinking is not supported on Claude Sonnet 5 and returns a 400. A question asking for the positive set cannot be answered from a single negative.
- The model scope is inside the anchor, so only the question changes.

REPAIRS MADE

- **question rewritten**

was: Where is `budget_tokens` supported?

now: Is manual extended thinking with `budget_tokens` supported on Claude Sonnet 5?

GOLD-B005-09 · NEEDS_REPAIR

openai · Human-in-the-loop · Human-in-the-loop › Marking tools that need approval

reasoning type `error_behavior` (generated as `configuration_interaction`) · shape `single_span` · precheck holdout-ready True

Q. What happens if the arguments are malformed JSON, are valid JSON but not an object (for example, `null` or a list), or contain non-standard constants such as `NaN`, `Infinity`, or `-Infinity`?

A. The callable is not invoked and the call requires manual approval.

ATOMIC CLAIMS

1. Callable approval rules fail closed when the SDK cannot safely inspect the arguments.
2. If the arguments are malformed JSON, are valid JSON but not an object (for example, `null` or a list), or contain non-standard constants such as `NaN`, `Infinity`, or `-Infinity`, the callable is not invoked and the call requires manual approval.

EXACT EVIDENCE

E1 · VER_AE3BFCC42C733C5051ABDA30F0F6DB07 1523–1855 (332 CHARS) · HUMAN-IN-THE-LOOP › MARKING TOOLS THAT NEED APPROVAL

Callable approval rules fail closed when the SDK cannot safely inspect the arguments. If the arguments are malformed JSON, are valid JSON but not an object (for example, `null` or a list), or contain non-standard constants such as `NaN`, `Infinity`, or `-Infinity`, the callable is not invoked and the call requires manual approval.

critical strings: `Callable approval rules fail closed`, `NaN`, `manual approval`

evidence_hash `d4a15c400fe14c77b39eb20fda908baeb1827d2ae89d4610af0994eee5b6fe7d`

INTERNAL REVIEW FINDINGS

- §9 satisfied on the conditions: all three — malformed JSON, valid JSON that is not an object, and non-standard constants — are disjuncts of a single conditional with one stated result, so nothing needs splitting or narrowing.
- CATEGORY: labelled `configuration_interaction`, but §4's three fields cannot be filled. The critical strings are `null`, `NaN` and `Infinity`, which are values rather than settings, and no second setting interacts with a first. Relabelled to `error_behavior`.
- CRITICAL_ANAPHORA: the answer's subject is 'the callable', whose antecedent — 'Callable approval rules fail closed when the SDK cannot safely inspect the arguments' — sits immediately before the anchor. Without it a reader cannot tell which callable is not invoked.

REPAIRS MADE

- **E1 anchor extended** (evidence_boundary_completion)

was 1609–1855, hash `25ef4cc6cccc83ffa6680f320e0f0538e39e36c44f1032f28b31d3be023653ec`

now 1523–1855, hash `d4a15c400fe14c77b39eb20fda908baeb1827d2ae89d4610af0994eee5b6fe7d`

- **atomic_claims rewritten**

was: ['If the arguments are malformed JSON, are valid JSON but not an object (for example, `null` or a list), or contain non-standard constants such as `NaN`, `Infinity`, or `-Infinity`, the callable is not invoked and the call requires manual approval.']

now: ['Callable approval rules fail closed when the SDK cannot safely inspect the arguments.', 'If the arguments are malformed JSON, are valid JSON but not an object (for example, `null` or a list), or contain non-standard constants such as `NaN`, `Infinity`, or `-Infinity`, the callable is not invoked and the call requires manual approval.']

- **reasoning_type rewritten**

was: `configuration_interaction`

now: `error_behavior`

- **secondary_category rewritten**

was: `conditional_behavior`

now: fail_closed_behavior

GOLD-B005-10 · REJECT_RECOMMENDED

openai · Models · Models › OpenAI models › Hosted multi-agent (experimental) › Current limitations
reasoning type `configuration_interaction` · shape `single_span` · precheck holdout-ready True

Q. What does `betas` override?

A. The experimental model rejects `reasoning.summary`, `max_tool_calls`, and caller-supplied `multi_agent` or `betas` overrides.

ATOMIC CLAIMS

1. The experimental model rejects `reasoning.summary`, `max_tool_calls`, and caller-supplied `multi_agent` or `betas` overrides.

EXACT EVIDENCE

E1 · VER_AE909BF8B4BBBE1D1A11119447F7AC94 19331–19456 (125 CHARS) · MODELS › OPENAI MODELS › HOSTED MULTI-AGENT (EXPERIMENTAL) › CURRENT LIMITATIONS

The experimental model rejects ``reasoning.summary``, ``max_tool_calls``, and caller-supplied ``multi_agent`` or ``betas`` overrides.

critical strings: `betas`, `reasoning.summary`, `max_tool_calls`
evidence_hash 540a39028df8184945b1d598976982b6092e1c52dbb919d3c009f6d5df2ccad0

INTERNAL REVIEW FINDINGS

- **RELATION_DIRECTION:** §10's suspicion is confirmed. The source says 'The experimental model rejects `reasoning.summary`, `max_tool_calls`, and caller-supplied `multi_agent` or `betas` overrides.' `betas` is the object of the rejection, not the subject of an override. The question asks what `betas` overrides, which the source never says.
- **SCOPE_IN_HEADING:** rewriting around the true subject would make the question about 'the experimental model', which the span identifies only by that phrase. The model's name sits in the section heading and, further down, after the span. There is no minimal outward expansion that brings the identity inside the anchor.
- Both defects would have to be repaired at once, and the second has no honest repair, so the candidate is recommended for rejection rather than rewritten.

GOLD-B005-11 · NEEDS REPAIR

openai · OpenAI Python API library · configured through AWS_REGION, AWS_DEFAULT_REGION, or your AWS profile.
reasoning type configuration_interaction · shape single_span · precheck holdout-ready True

Q. When using the `bedrock(...)` provider, what does setting `AWS_BEDROCK_BASE_URL` change?

A. Pass `base_url` to `bedrock(...)` or set `AWS_BEDROCK_BASE_URL` to override the derived `https://bedrock-mantle.<region>.api.aws/openai/v1` endpoint.

ATOMIC CLAIMS

1. Pass `base_url` to `bedrock(...)` or set `AWS_BEDROCK_BASE_URL` to override the derived `https://bedrock-mantle.<region>.api.aws/openai/v1` endpoint.

INTERACTION RECORDED

- **A:** `AWS_BEDROCK_BASE_URL`, or `base_url` passed to `bedrock(...)`
- **B:** the endpoint derived from the configured AWS region
- **relation:** A overrides B

EXACT EVIDENCE

E1 · VER_9247E3CE4DF6F79D9CADC44E1A3BBD0C 33783-33932 (149 CHARS) · CONFIGURED THROUGH AWS_REGION, AWS_DEFAULT_REGION, OR YOUR AWS PROFILE.

```
Pass `base_url` to `bedrock(...)` or set `AWS_BEDROCK_BASE_URL` to override the derived `https://bedrock-mantle.<region>.api.aws/openai/v1` endpoint.
```

critical strings: `AWS_BEDROCK_BASE_URL`, `base_url`

evidence_hash d48685e50739421736a63a780d66187b87e069d5b77514a0efa840fb92b35f55

INTERNAL REVIEW FINDINGS

- **DIRECTION_CORRECT:** unlike B005-10 the source does put the environment variable on the acting side — 'set `AWS_BEDROCK_BASE_URL` to override the derived ... endpoint'.
- **MISSING_CONDITION:** §11's warning applies. What is overridden is the endpoint *derived* from the region configuration, and the span says so; the question as asked implies an unconditional override. The provider that derives it, `bedrock(...)`, is named in the span.
- **SECTION_PATH_ARTEFACT:** the record's section_path is 'configured through AWS_REGION, AWS_DEFAULT_REGION, or your AWS profile.', which is prose the parser mistook for a heading. It is metadata only — no claim rests on it — but a reviewer should not read it as a real section.

REPAIRS MADE

- **question rewritten**
was: What does `AWS_BEDROCK_BASE_URL` override?
now: When using the `bedrock(...)` provider, what does setting `AWS_BEDROCK_BASE_URL` change?

GOLD-B005-12 · READY_FOR_OWNER_REVIEW

openai · Sandbox clients · Sandbox clients › Supported hosted platforms › Size Modal sandboxes
reasoning type configuration_interaction · shape single_span · precheck holdout-ready True

Q. What does FuseMountPattern require?

A. FuseMountPattern requires broad acknowledgement because blobfuse2 discovers ambient Azure authority, even when no inline credential is configured.

ATOMIC CLAIMS

1. FuseMountPattern requires broad acknowledgement because blobfuse2 discovers ambient Azure authority, even when no inline credential is configured.

INTERACTION RECORDED

- A: FuseMountPattern
- B: broad acknowledgement of the authority exposure
- relation: A requires B, because blobfuse2 discovers ambient Azure authority even with no inline credential

EXACT EVIDENCE

E1 · VER_3D4B8881962381CBFBA18ADE50C598E1 11024-11174 (150 CHARS) · SANDBOX CLIENTS ›
SUPPORTED HOSTED PLATFORMS › SIZE MODAL SANDBOXES

```
`FuseMountPattern` requires broad acknowledgement because `blobfuse2` discovers ambient Azure authority,  
even when no inline credential is configured.
```

critical strings: FuseMountPattern, blobfuse2

evidence_hash 6d34d63eb16fc99d04e20f8b9f024eff75fe086315cadca54dc94b3b88878a73

INTERNAL REVIEW FINDINGS

- §12 satisfied: FuseMountPattern is the grammatical subject of the requirement, the span states a requirement rather than a definition, and the reason — blobfuse2 discovering ambient Azure authority — is inside the anchor.

GOLD-B005-13 · REJECT_RECOMMENDED

openai · Sandbox clients · Sandbox clients › Supported hosted platforms › Size Modal sandboxes
reasoning type `configuration_interaction` · shape `single_span` · precheck holdout-ready True

Q. What does `S3FilesMountPattern` require?

A. `S3FilesMountPattern` likewise requires broad acknowledgement because `mount.s3files` uses ambient IAM authority.

ATOMIC CLAIMS

1. `S3FilesMountPattern` likewise requires broad acknowledgement because `mount.s3files` uses ambient IAM authority.

EXACT EVIDENCE

E1 · VER_3D4B8881962381CBFBA18ADE50C598E1 11175–11288 (113 CHARS) · SANDBOX CLIENTS ›
SUPPORTED HOSTED PLATFORMS › SIZE MODAL SANDBOXES

```
`S3FilesMountPattern` likewise requires broad acknowledgement because `mount.s3files` uses ambient IAM authority.
```

critical strings: `S3FilesMountPattern`, `mount.s3files`

evidence_hash b7cd45d15c27feeafd00ab13cda6f7664f164f3864fb5a9e0cb377e4f5f58c95

INTERNAL REVIEW FINDINGS

- DUPLICATE_RELATION: this is the sentence immediately after B005-12's, stating the same relation about a sibling mount pattern. §26 rejects 'the same configuration relation reworded', and two adjacent sentences from one paragraph are not two facts worth two benchmark cases.
- ANAPHORA: the span opens '`S3FilesMountPattern` likewise requires...', and 'likewise' refers to the `FuseMountPattern` sentence outside the anchor. The requirement is stated in full regardless, so this is noncritical — but combined with the duplication there is no reason to keep it.
- Keeping B005-12 and dropping this one is the choice; they are interchangeable.

GOLD-B005-14 · READY_FOR_OWNER_REVIEW

openai · Tools · Tools › Local runtime tools
reasoning type configuration_interaction · shape single_span · precheck holdout-ready True

Q. What does ApplyPatchTool require?

A. ComputerTool and ApplyPatchTool always require local implementations that you provide.

ATOMIC CLAIMS

- 1. ComputerTool and ApplyPatchTool always require local implementations that you provide.

INTERACTION RECORDED

- A: ComputerTool and ApplyPatchTool
- B: a local implementation supplied by the application
- relation: A always requires B

EXACT EVIDENCE

E1 · VER_CBEB36B7CF9A5E241940A011629B6F1B 14491-14581 (90 CHARS) · TOOLS › LOCAL RUNTIME TOOLS

`ComputerTool` and `ApplyPatchTool` always require local implementations that you provide.

critical strings: ApplyPatchTool, ComputerTool
evidence_hash b99555629d4f3bc24af3d059ad3a0b08f1eb616106369557e8228d4a115359fc

INTERNAL REVIEW FINDINGS

- §12 satisfied: ApplyPatchTool is a subject of the requirement (compound with ComputerTool), the span states a requirement, and 'local implementations that you provide' is operationally meaningful.

GOLD-B005-15 · NEEDS_REPAIR

openai · Tools · Tools › Local runtime tools › ComputerTool and the Responses computer tool
reasoning type `configuration_interaction` · shape `single_span` · precheck holdout-ready True

Q. When a `ComputerTool` is present, how are `tool_choice="computer"`, `"computer_use"` and `"computer_use_preview"` treated?

A. All three are accepted and normalized to the built-in selector that matches the effective request model.

ATOMIC CLAIMS

1. When a [`ComputerTool`][`agents.tool.ComputerTool`] is present, `tool_choice="computer"`, `"computer_use"`, and `"computer_use_preview"` are all accepted and normalized to the built-in selector that matches the effective request model.

INTERACTION RECORDED

- **A:** the presence of a `ComputerTool`
- **B:** the `tool_choice` value
- **relation:** A changes how B is interpreted: three otherwise-distinct strings are normalised to one built-in selector

EXACT EVIDENCE

E1 · VER_CBEB36B7CF9A5E241940A011629B6F1B 17432–17665 (233 CHARS) · TOOLS › LOCAL RUNTIME TOOLS › COMPUTERTOOL AND THE RESPONSES COMPUTER TOOL

```
When a [ComputerTool][agents.tool.ComputerTool] is present, tool_choice="computer", "computer_use", and "computer_use_preview" are all accepted and normalized to the built-in selector that matches the effective request model.
```

critical strings: `ComputerTool`, `tool_choice="computer"`, `"computer_use"`, `"computer_use_preview"`

evidence_hash 821cedbcd91b12d3dda4b832920ad3cc5756f02525eb06a621776aa1d97b85c7

INTERNAL REVIEW FINDINGS

- **QUESTION_BREADTH:** §13's concern. 'What happens when a `ComputerTool` is present?' invites any consequence; the span supports exactly one — three `tool_choice` strings are accepted and normalised to the selector matching the effective model.
- **LINK_MARKUP:** the question carries a raw markdown reference, '[`ComputerTool`][`agents.tool.ComputerTool`]', which the generator's link stripper did not reach. A question is not a place for documentation plumbing.

REPAIRS MADE

- **question rewritten**

was: What happens when a [`ComputerTool`][`agents.tool.ComputerTool`] is present?

now: When a `ComputerTool` is present, how are `tool_choice="computer"`, `"computer_use"` and `"computer_use_preview"` treated?

- **answer rewritten**

was: `tool_choice="computer"`, `"computer_use"`, and `"computer_use_preview"` are all accepted and normalized to the built-in selector that matches the effective request model.

now: All three are accepted and normalized to the built-in selector that matches the effective request model.

GOLD-B005-16 · NEEDS_REPAIR

openai · Usage · Usage › Accessing usage with sessions

reasoning type `configuration_interaction` · shape `single_span` · precheck holdout-ready True

Q. When you use a `Session` such as `SQLiteSession`, what usage does each call to `Runner.run(...)` return?

A. Usage for that specific run.

ATOMIC CLAIMS

1. When you use a `Session` (e.g., `SQLiteSession`), each call to `Runner.run(...)` returns usage for that specific run.

INTERACTION RECORDED

- **A:** using a `Session` (for example `SQLiteSession`)
- **B:** the usage returned by each `Runner.run(...)` call
- **relation:** A scopes B to the individual run rather than to the accumulated session

EXACT EVIDENCE

E1 · VER_F8002FE268B970EAEA8D640F9DD91FB3 4684-4801 (117 CHARS) · USAGE › ACCESSING USAGE WITH SESSIONS

When you use a `Session` (e.g., `SQLiteSession`), each call to `Runner.run(...)` returns usage for that specific run.

critical strings: `Session`, `SQLiteSession`

evidence_hash 40b53a39a3051bb1c73d88719255085ef89a539e53d19db5f6c5e7303eb61dcb

INTERNAL REVIEW FINDINGS

- **QUESTION_BREADTH:** §14's concern. 'What happens when you use a `Session`?' is broader than the span, which establishes one specific effect on usage accounting.
- The condition is preserved in the rewrite; only the consequence being asked about is named.

REPAIRS MADE

- **question rewritten**

was: What happens when you use a `Session` (e.g., `SQLiteSession`)?

now: When you use a `Session` such as `SQLiteSession`, what usage does each call to `Runner.run(...)` return?

- **answer rewritten**

was: Each call to `Runner.run(...)` returns usage for that specific run.

now: Usage for that specific run.

GOLD-B005-17 · READY_FOR_OWNER_REVIEW

openai · Running agents · Running agents › Runner lifecycle and configuration › The agent loop
reasoning type `error_behavior` · shape `single_span` · precheck holdout-ready True

Q. What happens if we exceed the `max_turns` passed?

A. We raise a `MaxTurnsExceeded` exception.

ATOMIC CLAIMS

1. If we exceed the `max_turns` passed, we raise a [`MaxTurnsExceeded`][`agents.exceptions.MaxTurnsExceeded`] exception.

EXACT EVIDENCE

E1 · VER_2C60E99CFD929A738910B893FD6F1A40 1936–2051 (115 CHARS) · RUNNING AGENTS › RUNNER LIFECYCLE AND CONFIGURATION › THE AGENT LOOP

If we exceed the ``max_turns`` passed, we raise a [``MaxTurnsExceeded``][`agents.exceptions.MaxTurnsExceeded`] exception.

critical strings: `max_turns`, `MaxTurnsExceeded`

evidence_hash `f1011d934522b2fe05de267455898fadd037e2ef0af581c5d11a7dd7571ea38`

INTERNAL REVIEW FINDINGS

- S6 satisfied: the condition (exceeding the `max_turns` passed) and the result (a `MaxTurnsExceeded` exception) are both inside the anchor, with no antecedent outside it.

GOLD-B005-18 · NEEDS_REPAIR

openai · OpenAI TypeScript and JavaScript API Library · OpenAI TypeScript and JavaScript API Library › Advanced Usage › Fetch options › Configuring proxies
reasoning type `exact_lookup` · shape `single_span` · precheck holdout-ready True

Q. What must an Undici-specific option like `dispatcher` be paired with?

A. The matching `fetch` implementation.

ATOMIC CLAIMS

1. Undici-specific options like `dispatcher` must be paired with the matching `fetch` implementation.

EXACT EVIDENCE

E1 · VER_F30A6447E4DF2AB76E4C1475F353109C 24178–24276 (98 CHARS) · OPENAI TYPESCRIPT AND JAVASCRIPT API LIBRARY › ADVANCED USAGE › FETCH OPTIONS › CONFIGURING PROXIES

Undici-specific options like `dispatcher` must be paired with the matching `fetch` implementation.

critical strings: `dispatcher`, `fetch`

evidence_hash 36d9e40ac1ce3cc82a00c5db4d11c8c00c3758313df7d54a8485e5759ad9f73c

INTERNAL REVIEW FINDINGS

- QUESTION_FORM: 'What must `dispatcher` be?' asks for a value. The span states a pairing requirement — an Undici-specific option must accompany the matching `fetch` implementation — so the question asks for something the evidence does not give.
- SCOPE: 'Undici-specific' is inside the anchor and belongs in the question; the section heading is not needed.

REPAIRS MADE

- **question rewritten**
was: What must `dispatcher` be?
now: What must an Undici-specific option like `dispatcher` be paired with?
- **answer rewritten**
was: Undici-specific options like `dispatcher` must be paired with the matching `fetch` implementation.
now: The matching `fetch` implementation.

GOLD-B005-19 · READY_FOR_OWNER_REVIEW

openai · Migration guide · Migration guide › Breaking changes › Removed `httpAgent` in favor of `fetchOptions`
reasoning type `lifecycle_compatibility_migration` · shape `single_span` · precheck holdout-ready True

Q. What happened to `httpAgent` ?

A. The `httpAgent` client option has been removed in favor of a platform-specific `fetchOptions` property.

ATOMIC CLAIMS

1. The `httpAgent` client option has been removed in favor of a [platform-specific `fetchOptions` property] (<https://github.com/openai/openai-node#fetch-options>).

EXACT EVIDENCE

E1 · VER_E8A7B17B5AF64679CADEA33CD8F6D250 10081–10239 (158 CHARS) · MIGRATION GUIDE › BREAKING CHANGES › REMOVED `HTTPAGENT` IN FAVOR OF `FETCHOPTIONS`

The `httpAgent` client option has been removed in favor of a [platform-specific `fetchOptions` property] (<https://github.com/openai/openai-node#fetch-options>).

critical strings: `httpAgent`, `fetchOptions`

evidence_hash 50dce494bcf32c01963ddf4bd2ddf4eebbe4659b793fcf5a7c5f5dbc95543201

INTERNAL REVIEW FINDINGS

- Self-contained removal statement naming both the removed option and its replacement. Narrowest accurate type is removal, which the secondary category records.

4. How a repair is gated

Every anchor repair is a strict outward growth of the span it replaces — an anchor that moves rather than grows is a different claim wearing the same candidate id — and both hashes are recorded. A repaired candidate is approved by quoting the post-repair hash, and the importer refuses an approval quoting the pre-repair one.

The generation artifact is not rewritten. `gold_review_batch_005.json` is byte-identical to what the generator produced; repairs live beside it in `gold_review_batch_005_repairs.json` with the original text, offsets and hashes preserved.

5. Invariants

- No new multi-hop search was run. Batch 005's result stands: 3 dependency-first pairs considered, 1 valid chain, 0 new unique chains.
- The semantic bridge-equivalence check is unchanged and still refuses `max_tokens` as a bridge — a request parameter in one span, a `stop_reason` value in the other.
- No retrieval system was run. `retrieval_was_not_run = true`; `systems_executed = []`.
- SYSTEM-A `9afcb5b7...` and SYSTEM-B `304c3509...` remain frozen and unexecuted.
- The holdout is not frozen (`holdout_frozen = false`).
- Batches 001–004 are byte-identical and their closure hashes still verify.

Generated by `scripts/build_batch_005_review_pdf.py` from `gold_review_batch_005.json`, `gold_review_batch_005_repairs.json`, `gold_batch_005_qc.json`, `human_decisions_batch_005.json` and `GOLD-001-eligibility-status.json`. Every figure is read from those artifacts at build time, and the build refuses to run if the repairs were computed against a different batch, if the packet claims any verified or eligible candidate, or if a decision has already been recorded. Batch 005, schema 1.3, corpus snapshot snap_689e336380a054d8039dc35b2c09cd0a, batch_sha256 37bf3509a9205637588730760dc14bf2..., reviewed 2026-08-23T16:42:37Z by claude (internal authoring review). Raw provider documentation is not redistributed; quoted spans are the short excerpts under review.